SRI CHANDRASEKHARENDRA SARASWATHI VISHWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Name : Keerthiga G T
Reg. No : 11249A162
Class : III B.E.(CSE)
Course Code :
Course Name : Database Management System

Index of Experiments

Exp. No.	Title	Database / Schema
1	Creation of a Database and SQL Queries for Information Retrieval	UniversityDB
2	Insertion, Deletion, Modification, Alteration, Updating and Conditional Record Viewing	PayrollLabDB
3	DCL and TCL Commands	BankDB
4	Creating a Database to Set Various Constraints	RetailConstraintDB
5	Advanced SQL Queries and Set Operators	LibraryExp5
6	SQL Aggregate Functions, GROUP BY and HAVING	RetailSalesDB
7	Creation of Views, Sequences, Indexes and Savepoints	WarehouseDB
8	Normalization (1NF, 2NF, 3NF, 4NF and 5NF) using Dependencies	NormalizationLabDB
9	Creating Relationships Between Tables and Performing JOINs and Subqueries	CompanyDB
10	PL/SQL Programming: Conditional Blocks, Exceptions, Procedures, Functions, Triggers and Cursors	Oracle schema

Experiment 1

Creation of a Database and SQL Queries for Information Retrieval

Aim: To create a relational database and the STUDENT table, insert ten records, and retrieve information using SQL filtering, sorting, pattern matching and conditional operators.

Platform: MySQL 8.0+

Query 1: Create and Select Database

Query:

```
CREATE DATABASE UniversityDB;

USE UniversityDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create Student Table

Query:

```
CREATE TABLE Student (
  Student_ID INT PRIMARY KEY,
  Student_Name VARCHAR(50) NOT NULL,
  Department VARCHAR(20) NOT NULL,
  Year INT NOT NULL,
  Age INT,
  Gender VARCHAR(10),
  Marks DECIMAL(5,2),
  City VARCHAR(30)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Insert 10 Records

Query:

```
INSERT INTO Student
  (Student_ID, Student_Name, Department, Year, Age, Gender, Marks, City)
VALUES
  (101, 'Arun', 'CSE', 2, 19, 'Male', 85, 'Chennai'),
  (102, 'Priya', 'ECE', 3, 20, 'Female', 92, 'Hyderabad'),
  (103, 'Ravi', 'CSE', 2, 19, 'Male', 67, 'Vijayawada'),
  (104, 'Sneha', 'AIML', 1, 18, 'Female', 88, 'Bengaluru'),
  (105, 'Kiran', 'EEE', 4, 21, 'Male', 56, 'Chennai'),
  (106, 'Divya', 'CSE', 3, 20, 'Female', 94, 'Tirupati'),
  (107, 'Rahul', 'MECH', 2, 19, 'Male', 72, 'Nellore'),
  (108, 'Anjali', 'AIML', 1, 18, 'Female', 81, 'Hyderabad'),
  (109, 'Naveen', 'ECE', 4, 21, 'Male', 63, 'Guntur'),
  (110, 'Kavya', 'CSE', 3, 20, 'Female', 90, 'Chennai');
```

Output:

```
Query OK, 10 rows affected  Records: 10  Duplicates: 0  Warnings: 0
```

Query 4: Display Table Structure

Query:

```
DESC
Student;
```

Output:

Field	Type	Null	Key	Default	Extra
Student_ID	int	NO	PRI	NULL	
Student_Name	varchar(50)	NO		NULL	
Department	varchar(20)	NO		NULL	
Year	int	NO		NULL	
Age	int	YES		NULL	
Gender	varchar(10)	YES		NULL	
Marks	decimal(5,2)	YES		NULL	
City	varchar(30)	YES		NULL	

Query 5: Display All Records

Query:
```
SELECT * FROM
Student ORDER BY
Student_ID;
```
Output:

Student_ID	Student_Name	Department	Year	Age	Gender	Marks	City
101	Arun	CSE	2	19	Male	85.00	Chennai
102	Priya	ECE	3	20	Female	92.00	Hyderabad
103	Ravi	CSE	2	19	Male	67.00	Vijayawada
104	Sneha	AIML	1	18	Female	88.00	Bengaluru
105	Kiran	EEE	4	21	Male	56.00	Chennai
106	Divya	CSE	3	20	Female	94.00	Tirupati
107	Rahul	MECH	2	19	Male	72.00	Nellore
108	Anjali	AIML	1	18	Female	81.00	Hyderabad
109	Naveen	ECE	4	21	Male	63.00	Guntur
110	Kavya	CSE	3	20	Female	90.00	Chennai

Query 6: Display Selected Columns

Query:
```
SELECT Student_ID, Student_Name, Department
FROM Student
ORDER BY Student_ID;
```
Output:

Student_ID	Student_Name	Department
101	Arun	CSE
102	Priya	ECE
103	Ravi	CSE
104	Sneha	AIML

105	Kiran	EEE
106	Divya	CSE
107	Rahul	MECH
108	Anjali	AIML
109	Naveen	ECE
110	Kavya	CSE

Query 7: Display CSE Students

Query:

```
SELECT Student_ID, Student_Name, Department
FROM Student WHERE
Department = 'CSE'
ORDER BY Student_ID;
```

Output:

Student_ID	Student_Name	Department
101	Arun	CSE
103	Ravi	CSE
106	Divya	CSE
110	Kavya	CSE

Query 8: AND Operator

Query:

```
SELECT Student_Name, Department, Marks
FROM Student
WHERE Department = 'CSE' AND Marks > 80
ORDER BY Student_ID;
```

Output:

Student_Name	Department	Marks
Arun	CSE	85.00
Divya	CSE	94.00
Kavya	CSE	90.00

Query 9: OR Operator

Query:

```
SELECT Student_Name, Department
FROM Student
WHERE Department = 'AIML' OR Department = 'ECE'
ORDER BY Student_ID;
```

Output:

Student_Name	Department
Priya	ECE
Sneha	AIML
Anjali	AIML

Naveen	ECE

Query 10: DISTINCT Operator

Query:

```
SELECT DISTINCT Department
FROM Student
ORDER BY Department;
```

Output:

Department
AIML
CSE
ECE
EEE
MECH

Query 11: Ascending Order

Query:

```
SELECT Student_Name, Marks
FROM Student
ORDER BY Marks ASC;
```

Output:

Student_Name	Marks
Kiran	56.00
Naveen	63.00
Ravi	67.00
Rahul	72.00
Anjali	81.00
Arun	85.00
Sneha	88.00
Kavya	90.00
Priya	92.00
Divya	94.00

Query 12: Descending Order

Query:

```
SELECT Student_Name, Marks
FROM Student
ORDER BY Marks DESC;
```

Output:

Student_Name	Marks
Divya	94.00

Priya	92.00
Kavya	90.00
Sneha	88.00
Arun	85.00
Anjali	81.00
Rahul	72.00
Ravi	67.00
Naveen	63.00
Kiran	56.00

Query 13: BETWEEN Operator

Query:

```
SELECT Student_Name, Marks
FROM Student WHERE Marks
BETWEEN 70 AND 90
ORDER BY Marks ASC;
```

Output:

Student_Name	Marks
Rahul	72.00
Anjali	81.00
Arun	85.00
Sneha	88.00
Kavya	90.00

Query 14: LIKE Operator

Query:

```
SELECT Student_ID, Student_Name
FROM Student WHERE
Student_Name LIKE 'A%'
ORDER BY Student_ID;
```

Output:

Student_ID	Student_Name
101	Arun
108	Anjali

Query 15: IN Operator

Query:

```
SELECT Student_Name, Department
FROM Student
WHERE Department IN ('CSE', 'ECE', 'AIML')
ORDER BY Student_ID;
```

Output:

Student_Name	Department

Arun	CSE
Priya	ECE
Ravi	CSE
Sneha	AIML
Divya	CSE
Anjali	AIML
Naveen	ECE
Kavya	CSE

Query 16: IS NULL Operator

Query:

```
SELECT Student_ID, Student_Name, City
FROM Student
WHERE City IS NULL;
```

Output:

```
Empty set (0 rows)
```

Query 17: Display Top 3 Students

Query:

```
SELECT Student_Name, Department, Marks
FROM Student
ORDER BY Marks DESC, Student_ID ASC
LIMIT 3;
```

Output:

Student_Name	Department	Marks
Divya	CSE	94.00
Priya	ECE	92.00
Kavya	CSE	90.00

Query 18: Count Total Records

Query:

```
SELECT COUNT(*) AS Total
FROM Student;
```

Output:

Total
10

Query 19: Count Distinct Departments

Query:

```
SELECT COUNT(DISTINCT Department) AS Departments
FROM Student;
```

Output:

Departments
5

Query 20: Highest and Lowest Marks

Query:

```
SELECT MAX(Marks) AS Highest, MIN(Marks) AS Lowest FROM Student;
```

Output:

Highest	Lowest
94.00	56.00

Experiment 2

Insertion, Deletion, Modification, Alteration, Updating and Conditional Record Viewing

Aim: To create an Employee Payroll database and demonstrate INSERT, DELETE, ALTER, UPDATE and SELECT statements, including conditional modifications and retrieval.

Platform: MySQL 8.0+

Query 1: Create and Select Database

Query:

```
CREATE DATABASE PayrollLabDB;

USE PayrollLabDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create Employee Table

Query:

```
CREATE TABLE Employee (
  Emp_ID INT PRIMARY KEY,
  Emp_Name VARCHAR(50) NOT NULL,
  Department VARCHAR(25) NOT NULL,
  Designation VARCHAR(30) NOT NULL,
  Salary DECIMAL(10,2) NOT NULL,
  City VARCHAR(30),
  Join_Date DATE NOT NULL
);
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Insert Eight Records

Query:

```
INSERT INTO Employee VALUES
  (201, 'Asha', 'HR', 'Executive', 38000, 'Chennai', '2023-06-12'),
  (202, 'Bharat', 'IT', 'Developer', 52000, 'Hyderabad', '2022-08-01'),
  (203, 'Charan', 'Sales', 'Associate', 32000, 'Vijayawada', '2024-01-15'),
  (204, 'Deepa', 'IT', 'Analyst', 48000, 'Bengaluru', '2023-03-20'),
  (205, 'Eshan', 'Finance', 'Accountant', 45000, 'Chennai', '2021-11-05'),
  (206, 'Farah', 'HR', 'Recruiter', 41000, 'Hyderabad', '2024-02-10'),
  (207, 'Gopal', 'Sales', 'Executive', 35000, 'Nellore', '2022-07-18'),
  (208, 'Hema', 'IT', 'Developer', 56000, 'Tirupati', '2021-09-30');
```

Output:

```
Query OK, 8 rows affected  Records: 8  Duplicates: 0  Warnings: 0
```

Query 4: Display All Records

Query:

```
SELECT * FROM Employee
ORDER BY Emp_ID;
```

Output:

Emp_ID	Emp_Name	Department	Designation	Salary	City	Join_Date
201	Asha	HR	Executive	38000.00	Chennai	2023-06-12

202	Bharat	IT	Developer	52000.00	Hyderabad	2022-08-01
203	Charan	Sales	Associate	32000.00	Vijayawada	2024-01-15
204	Deepa	IT	Analyst	48000.00	Bengaluru	2023-03-20
205	Eshan	Finance	Accountant	45000.00	Chennai	2021-11-05
206	Farah	HR	Recruiter	41000.00	Hyderabad	2024-02-10
207	Gopal	Sales	Executive	35000.00	Nellore	2022-07-18
208	Hema	IT	Developer	56000.00	Tirupati	2021-09-30

Query 5: Insert New Employee

Query:

```
INSERT INTO Employee VALUES
  (209, 'Ishita', 'Marketing', 'Coordinator',
  39000, 'Guntur', '2025-01-06');

SELECT Emp_ID, Emp_Name, Department, Salary
FROM Employee
WHERE Emp_ID = 209;
```

Output:

```
Query OK, 1 row affected
```

Emp_ID	Emp_Name	Department	Salary
209	Ishita	Marketing	39000.00

Query 6: Update Employee Salary

Query:

```
UPDATE Employee
SET Salary = 55000
WHERE Emp_ID = 202;

SELECT Emp_ID, Emp_Name, Salary
FROM Employee
WHERE Emp_ID = 202;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Emp_ID	Emp_Name	Salary
202	Bharat	55000.00

Query 7: Increase Sales Salary by 10%

Query:

```
UPDATE Employee
SET Salary = ROUND(Salary * 1.10, 2)
WHERE Department = 'Sales';

SELECT Emp_ID, Emp_Name, Salary
FROM Employee
WHERE Department = 'Sales'
ORDER BY Emp_ID;
```

Output:

```
Query OK, 2 rows affected  Rows matched: 2  Changed: 2  Warnings: 0
```

Emp_ID	Emp_Name	Salary
203	Charan	35200.00
207	Gopal	38500.00

Query 8: Modify Employee Department

Query:

```
UPDATE Employee
SET Department = 'Operations'
WHERE Emp_ID = 206;

SELECT Emp_ID, Emp_Name, Department
FROM Employee
WHERE Emp_ID = 206;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Emp_ID	Emp_Name	Department
206	Farah	Operations

Query 9: Add Email Column

Query:

```
ALTER TABLE Employee
ADD COLUMN Email VARCHAR(80) NULL;

SHOW COLUMNS FROM Employee LIKE 'Email';
```

Output:

```
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Field	Type	Null	Key	Default	Extra
Email	varchar(80)	YES		NULL	

Query 10: Update Email Addresses

Query:

```
UPDATE Employee
SET Email = 'asha@company.test'
WHERE Emp_ID = 201;

UPDATE Employee
SET Email = 'bharat@company.test'
WHERE Emp_ID = 202;

SELECT Emp_ID, Emp_Name, Email
FROM Employee
WHERE Email IS NOT NULL
ORDER BY Emp_ID;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Emp_ID	Emp_Name	Email
201	Asha	asha@company.test
202	Bharat	bharat@company.test

Query 11: Modify Column Data Type

Query:

```
ALTER TABLE Employee
MODIFY COLUMN Designation VARCHAR(50) NOT NULL;

SHOW COLUMNS FROM Employee LIKE 'Designation';
```

Output:

```
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Field	Type	Null	Key	Default	Extra
Designation	varchar(50)	NO		NULL	

Query 12: Display Employees with Salary ≥ 45000

Query:

```
SELECT Emp_ID, Emp_Name, Department, Salary
FROM Employee
WHERE Salary >= 45000
ORDER BY Salary DESC, Emp_ID;
```

Output:

Emp_ID	Emp_Name	Department	Salary
208	Hema	IT	56000.00
202	Bharat	IT	55000.00
204	Deepa	IT	48000.00
205	Eshan	Finance	45000.00

Query 13: Delete Employee 209

Query:

```
DELETE FROM Employee
WHERE Emp_ID = 209;

SELECT COUNT(*) AS Total_Employees
FROM Employee;
```

Output:

```
Query OK, 1 row affected
```

Total_Employees
8

Query 14: Delete Employee Using Condition

Query:

```
DELETE FROM Employee
WHERE Department = 'Operations'
```

```
AND Emp_ID = 206;

SELECT Emp_ID, Emp_Name, Department
FROM Employee
ORDER BY Emp_ID;
```

Output:

```
Query OK, 1 row affected
```

Emp_ID	Emp_Name	Department
201	Asha	HR
202	Bharat	IT
203	Charan	Sales
204	Deepa	IT
205	Eshan	Finance
207	Gopal	Sales
208	Hema	IT

Query 15: Display Final Payroll

Query:

```
SELECT Emp_ID, Emp_Name, Department, Salary, Email
FROM Employee
ORDER BY Emp_ID;
```

Output:

Emp_ID	Emp_Name	Department	Salary	Email
201	Asha	HR	38000.00	asha@company.test
202	Bharat	IT	55000.00	bharat@company.test
203	Charan	Sales	35200.00	NULL
204	Deepa	IT	48000.00	NULL
205	Eshan	Finance	45000.00	NULL
207	Gopal	Sales	38500.00	NULL
208	Hema	IT	56000.00	NULL

Query 16: Final Employee Count

Query:

```
SELECT COUNT(*) AS Final_Count
FROM Employee;
```

Output:

Final_Count
7

Query 17: Sales Department Salaries

Query:

```
SELECT Emp_Name, Salary
```

```
FROM Employee
WHERE Department = 'Sales'
ORDER BY Emp_ID;
```

Output:

Emp_Name	Salary
Charan	35200.00
Gopal	38500.00

Query 18: Display Final Table Structure

Query:

```
SHOW COLUMNS FROM Employee;
```

Output:

Field	Type	Null	Key	Default	Extra
Emp_ID	int	NO	PRI	NULL	
Emp_Name	varchar(50)	NO		NULL	
Department	varchar(25)	NO		NULL	
Designation	varchar(50)	NO		NULL	
Salary	decimal(10,2)	NO		NULL	
City	varchar(30)	YES		NULL	
Join_Date	date	NO		NULL	
Email	varchar(80)	YES		NULL	

Experiment 3

DCL and TCL Commands

Aim: To create a banking database and demonstrate Data Control Language (DCL) operations for user permissions and Transaction Control Language (TCL) operations for safe, consistent banking updates using MySQL.

Platform: MySQL 8.0+

Query 1: Create and Select Database

Query:

```
CREATE DATABASE IF NOT EXISTS BankDB;

USE BankDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create BankAccount Table

Query:

```
CREATE TABLE BankAccount (
  Account_ID INT PRIMARY KEY,
  Holder_Name VARCHAR(50) NOT NULL,
  Branch VARCHAR(30) NOT NULL,
  Balance DECIMAL(12,2) NOT NULL CHECK (Balance >= 0),
  Account_Type VARCHAR(20) NOT NULL
) ENGINE=InnoDB;
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Insert Five Records

Query:

```
INSERT INTO BankAccount VALUES
  (201, 'Anand', 'Chennai', 50000, 'Savings'),
  (202, 'Bhavya', 'Hyderabad', 35000, 'Savings'),
  (203, 'Charan', 'Vijayawada', 20000, 'Current'),
  (204, 'Deepa', 'Chennai', 45000, 'Savings'),
  (205, 'Eshan', 'Bengaluru', 60000, 'Current');
```

Output:

```
Query OK, 5 rows affected  Records: 5  Duplicates: 0  Warnings: 0
```

Query 4: Display Account Balances

Query:

```
SELECT Account_ID, Holder_Name, Balance
FROM BankAccount
ORDER BY Account_ID;
```

Output:

Account_ID	Holder_Name	Balance
201	Anand	50000.00
202	Bhavya	35000.00
203	Charan	20000.00

204	Deepa	45000.00
205	Eshan	60000.00

Query 5: Create New User

Run as the administrator account.

Query:

```
CREATE USER 'lab_teller3'@'localhost'
IDENTIFIED BY 'LabDemo_ChangeMe_2026!';
```

Output:

```
Query OK, 0 rows affected
```

Query 6: GRANT Permission

Run as the administrator account.

Query:

```
GRANT SELECT ON BankDB.BankAccount
TO 'lab_teller3'@'localhost';
```

Output:

```
Query OK, 0 rows affected
```

Query 7: Display User Privileges

Run as the administrator account.

Query:

```
SHOW GRANTS FOR 'lab_teller3'@'localhost';
```

Output:

Grants for lab_teller3@localhost
GRANT USAGE ON *.* TO `lab_teller3`@`localhost`
GRANT SELECT ON `BankDB`.`BankAccount` TO `lab_teller3`@`localhost`

Query 8: Test User Permissions

Connect as the user lab_teller3 before running these statements. The SELECT succeeds; the UPDATE must be denied.

Query:

```
SELECT Account_ID, Balance
FROM BankDB.BankAccount;

UPDATE BankDB.BankAccount SET
Balance = Balance + 100
WHERE Account_ID = 201;
```

Output:

Account_ID	Balance
201	50000.00
202	35000.00
203	20000.00
204	45000.00
205	60000.00

```
ERROR 1142 (42000): UPDATE command denied to user 'lab_teller3'@'localhost' for
table 'BankAccount'
```

Query 9: REVOKE Permission

Run as the administrator account.

Query:

```
REVOKE SELECT ON BankDB.BankAccount
FROM 'lab_teller3'@'localhost';

SHOW GRANTS FOR 'lab_teller3'@'localhost';
```

Output:

```
Query OK, 0 rows affected
```

Grants for lab_teller3@localhost
GRANT USAGE ON *.* TO `lab_teller3`@`localhost`

Query 10: COMMIT Transaction

Query:

```
START TRANSACTION;
UPDATE BankAccount
SET Balance = Balance + 5000
WHERE Account_ID = 201;

COMMIT;

SELECT Account_ID, Balance
FROM BankAccount
WHERE Account_ID = 201;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Account_ID	Balance
201	55000.00

Query 11: ROLLBACK Transaction

Query:

```
START TRANSACTION;
UPDATE BankAccount
SET Balance = Balance - 3000
WHERE Account_ID = 202;

ROLLBACK;

SELECT Account_ID, Balance
FROM BankAccount
WHERE Account_ID = 202;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Account_ID	Balance
202	35000.00

Query 12: SAVEPOINT and Partial ROLLBACK

Query:

```
START TRANSACTION;
```

```
UPDATE BankAccount
SET Balance = Balance + 1000
WHERE Account_ID = 203;

SAVEPOINT after_first_change;

UPDATE BankAccount
SET Balance = Balance + 2000
WHERE Account_ID = 204;

ROLLBACK TO SAVEPOINT after_first_change;

COMMIT;

SELECT Account_ID, Balance
FROM BankAccount
WHERE Account_ID IN (203, 204)
ORDER BY Account_ID;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Account_ID	Balance
203	21000.00
204	45000.00

Query 13: Transfer Money Using COMMIT

Query:

```
START TRANSACTION;
UPDATE BankAccount
SET Balance = Balance - 4000
WHERE Account_ID = 201 AND Balance >= 4000;

SELECT ROW_COUNT() AS Debit_Rows;

UPDATE BankAccount
SET Balance = Balance + 4000
WHERE Account_ID = 202;

SELECT ROW_COUNT() AS Credit_Rows;

COMMIT;

SELECT Account_ID, Balance
FROM BankAccount
WHERE Account_ID IN (201, 202)
ORDER BY Account_ID;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Debit_Rows
1

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Credit_Rows
1

Account_ID	Balance
201	51000.00
202	39000.00

Query 14: RELEASE SAVEPOINT

Query:

```
START TRANSACTION;

SAVEPOINT test_point;

RELEASE SAVEPOINT test_point;

ROLLBACK;
```

Output:

```
Query OK, 0 rows affected
Query OK, 0 rows affected
Query OK, 0 rows affected
Query OK, 0 rows affected
```

Query 15: Delete Test User

Run as the administrator account.

Query:

```
DROP USER 'lab_teller3'@'localhost';
```

Output:

```
Query OK, 0 rows affected
```

Query 16: Verify Final Account Balances

Query:

```
SELECT Account_ID, Balance
FROM BankAccount
ORDER BY Account_ID;
```

Output:

Account_ID	Balance
201	51000.00
202	39000.00
203	21000.00
204	45000.00
205	60000.00

Query 17: Verify Total Balance

Query:

```
SELECT SUM(Balance) AS Total_Balance
FROM BankAccount;
```

Output:

Total_Balance

216000.00

Experiment 4

Creating a Database to Set Various Constraints

Aim: To design a relational database with PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK and DEFAULT constraints; insert valid data; and observe how MySQL rejects records that violate data integrity rules.

Platform: MySQL 8.0+

Query 1: Create Database

Query:

```
CREATE DATABASE IF NOT EXISTS RetailConstraintDB;
USE RetailConstraintDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create Customers Table

Query:

```
CREATE TABLE Customers (
customer_id INT PRIMARY KEY,
customer_name VARCHAR(60) NOT NULL,
email VARCHAR(80) NOT NULL UNIQUE,
city VARCHAR(40) DEFAULT 'Chennai'
) ENGINE=InnoDB;
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Create Products Table

Query:

```
CREATE TABLE Products (   product_id
INT PRIMARY KEY,   sku VARCHAR(20)
NOT NULL UNIQUE,   product_name
VARCHAR(60) NOT NULL,   price
DECIMAL(10,2) NOT NULL,   stock INT
NOT NULL DEFAULT 0,
  CONSTRAINT chk_price CHECK (price > 0),
  CONSTRAINT chk_stock CHECK (stock >= 0)
) ENGINE=InnoDB;
```

Output:

```
Query OK, 0 rows affected
```

Query 4: Create Orders Table

Query:

```
CREATE TABLE Orders (
order_id INT PRIMARY KEY,
customer_id INT NOT NULL,
product_id INT NOT NULL,
quantity INT NOT NULL,
  order_status VARCHAR(12) NOT NULL DEFAULT 'Pending',
order_date DATE NOT NULL DEFAULT (CURRENT_DATE),
  CONSTRAINT fk_order_customer
  FOREIGN KEY (customer_id)
  REFERENCES Customers(customer_id),
  CONSTRAINT fk_order_product
```

```
  FOREIGN KEY (product_id)
  REFERENCES Products(product_id),
  CONSTRAINT chk_quantity CHECK (quantity > 0),
CONSTRAINT chk_status CHECK (
    order_status IN ('Pending', 'Confirmed', 'Cancelled')
  )
) ENGINE=InnoDB;
```

Output:

```
Query OK, 0 rows affected
```

Query 5: Insert Customers

Query:

```
INSERT INTO Customers
  (customer_id, customer_name, email, city)
VALUES
  (201, 'Asha', 'asha@example.com', 'Chennai'),
  (202, 'Bala', 'bala@example.com', 'Hyderabad'),
  (203, 'Charu', 'charu@example.com', 'Bengaluru');
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 6: Insert Products

Query:

```
INSERT INTO Products
  (product_id, sku, product_name, price, stock)
VALUES
  (301, 'KB-01', 'Keyboard', 850, 25),
  (302, 'MS-02', 'Mouse', 450, 40),
  (303, 'HD-03', 'Hard Drive', 4200, 10);
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 7: Insert Orders

Query:

```
INSERT INTO Orders
  (order_id, customer_id, product_id, quantity, order_status)
VALUES
  (401, 201, 301, 2, 'Pending'),
  (402, 202, 302, 3, 'Confirmed'),
  (403, 203, 303, 1, 'Pending');
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 8: Test DEFAULT Constraint

Query:

```
INSERT INTO Customers
  (customer_id, customer_name, email)
VALUES
  (204, 'Deepa', 'deepa@example.com');

SELECT customer_id, customer_name, city
FROM Customers
WHERE customer_id = 204;
```

Output:

```
Query OK, 1 row affected
```

customer_id	customer_name	city
204	Deepa	Chennai

Query 9: Test PRIMARY KEY Constraint

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Customers
  (customer_id, customer_name, email)
VALUES
  (201, 'Duplicate', 'duplicate@example.com');
```

Output:

```
ERROR 1062 (23000): Duplicate entry '201' for key 'Customers.PRIMARY'
```

Query 10: Test UNIQUE Constraint

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Customers
  (customer_id, customer_name, email)
VALUES
  (205, 'Esha', 'asha@example.com');
```

Output:

```
ERROR 1062 (23000): Duplicate entry 'asha@example.com' for key 'Customers.email'
```

Query 11: Test NOT NULL Constraint

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Customers
  (customer_id, customer_name, email)
VALUES
  (205, NULL, 'esha@example.com');
```

Output:

```
ERROR 1048 (23000): Column 'customer_name' cannot be null
```

Query 12: Test CHECK Constraint (Price)

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Products
  (product_id, sku, product_name, price, stock)
VALUES
  (304, 'BAD-04', 'Invalid Product', -10, 5);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'chk_price' is violated.
```

Query 13: Test CHECK Constraint (Quantity)

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Orders
  (order_id, customer_id, product_id, quantity)
VALUES
  (404, 201, 301, 0);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'chk_quantity' is violated.
```

Query 14: Test FOREIGN KEY Constraint

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Orders
  (order_id, customer_id, product_id, quantity)
VALUES
  (404, 999, 301, 1);
```

Output:

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`RetailConstraintDB`.`Orders`, CONSTRAINT `fk_order_customer` FOREIGN KEY
(`customer_id`) REFERENCES `Customers` (`customer_id`))
```

Query 15: Test CHECK Constraint (Status)

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Orders
  (order_id, customer_id, product_id, quantity, order_status)
VALUES
  (404, 201, 301, 1, 'Shipped');
```

Output:

```
ERROR 3819 (HY000): Check constraint 'chk_status' is violated.
```

Query 16(a): Display Customers

Query:

```
SELECT customer_id, customer_name, email, city
FROM Customers
ORDER BY customer_id;
```

Output:

customer_id	customer_name	email	city
201	Asha	asha@example.com	Chennai
202	Bala	bala@example.com	Hyderabad
203	Charu	charu@example.com	Bengaluru
204	Deepa	deepa@example.com	Chennai

Query 16(b): Display Orders

Query:

```
SELECT order_id, customer_id, product_id,
quantity, order_status FROM Orders
ORDER BY order_id;
```

Output:

order_id	customer_id	product_id	quantity	order_status
401	201	301	2	Pending
402	202	302	3	Confirmed
403	203	303	1	Pending

Query 16(c): Display Table Constraints

Query:

```
SHOW CREATE TABLE Orders;
```

Output:

Table	Create Table
Orders	CREATE TABLE `Orders` (`order_id` int NOT NULL, `customer_id` int NOT NULL, `product_id` int NOT NULL, `quantity` int NOT NULL, `order_status` varchar(12) NOT NULL DEFAULT 'Pending', `order_date` date NOT NULL DEFAULT (curdate()), PRIMARY KEY (`order_id`), KEY `fk_order_customer` (`customer_id`), KEY `fk_order_product` (`product_id`), CONSTRAINT `fk_order_customer` FOREIGN KEY (`customer_id`) REFERENCES `Customers` (`customer_id`), CONSTRAINT `fk_order_product` FOREIGN KEY (`product_id`) REFERENCES `Products` (`product_id`), CONSTRAINT `chk_quantity` CHECK ((`quantity` > 0)), CONSTRAINT `chk_status` CHECK ((`order_status` in (_utf8mb4'Pending',_utf8mb4'Confirmed',_utf8mb4'Cancelled')))) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

Experiment 5

Advanced SQL Queries and Set Operators

Aim: To create related library tables and execute MySQL queries using ANY, ALL, IN, EXISTS, NOT EXISTS, UNION, INTERSECT and integrity constraints; and interpret the results against a known dataset.

Platform: MySQL 8.0+

Query 1: Create Database

Query:

```
CREATE DATABASE IF NOT EXISTS LibraryExp5;

USE LibraryExp5;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2(a): Create Books Table

Query:

```
CREATE TABLE Books (   book_id
INT PRIMARY KEY,   title
VARCHAR(80) NOT NULL,   category
VARCHAR(30) NOT NULL,   price
DECIMAL(8,2) NOT NULL,   copies
INT NOT NULL,   CHECK (price >=
0),
  CHECK (copies >= 0)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 2(b): Create Members Table

Query:

```
CREATE TABLE Members (   member_id
INT PRIMARY KEY,   member_name
VARCHAR(60) NOT NULL,   department
VARCHAR(20) NOT NULL
);
```

Output:

```
Query OK, 0 rows affected
```

Query 2(c): Create Loans Table

Query:

```
CREATE TABLE Loans (
loan_id INT PRIMARY KEY,
book_id INT NOT NULL,
member_id INT NOT NULL,
loan_date DATE NOT NULL,
  FOREIGN KEY (book_id) REFERENCES Books(book_id),
  FOREIGN KEY (member_id) REFERENCES Members(member_id) );
```

Output:

```
Query OK, 0 rows affected
```

Query 3(a): Insert Books

Query:

```
INSERT INTO Books VALUES
  (1, 'SQL Basics', 'Database', 400, 5),
  (2, 'Advanced SQL', 'Database', 650, 3),
  (3, 'Python Guide', 'Programming', 550, 4),
  (4, 'Java Handbook', 'Programming', 700, 2),
  (5, 'Data Science', 'Analytics', 900, 2),
  (6, 'Statistics 101', 'Analytics', 450, 6),
  (7, 'Web Design', 'Web', 300, 5),
  (8, 'Cloud Computing', 'Cloud', 800, 3);
```

Output:

```
Query OK, 8 rows affected  Records: 8  Duplicates: 0  Warnings: 0
```

Query 3(b): Insert Members

Query:

```
INSERT INTO Members VALUES
  (101, 'Asha', 'CSE'),
  (102, 'Bala', 'ECE'),
  (103, 'Charan', 'CSE'),
  (104, 'Deepa', 'AIML'),
  (105, 'Esha', 'MECH');
```

Output:

```
Query OK, 5 rows affected  Records: 5  Duplicates: 0  Warnings: 0
```

Query 3(c): Insert Loans

Query:

```
INSERT INTO Loans VALUES
  (201, 1, 101, '2026-09-01'),
  (202, 3, 101, '2026-09-02'),
  (203, 2, 102, '2026-09-03'),
  (204, 5, 104, '2026-09-04'),
  (205, 4, 103, '2026-09-05');
```

Output:

```
Query OK, 5 rows affected  Records: 5  Duplicates: 0  Warnings: 0
```

Query 4: IN Operator – Display Database or Analytics books

Query:

```
SELECT book_id, title
FROM Books
WHERE category IN ('Database',
'Analytics') ORDER BY book_id;
```

Output:

book_id	title
1	SQL Basics
2	Advanced SQL
5	Data Science
6	Statistics 101

Query 5: ANY Operator – Display books priced above at least one Database book

Query:

```
SELECT book_id, title, price
FROM Books
WHERE price > ANY (
  SELECT price
  FROM Books
  WHERE category = 'Database'
)
ORDER BY book_id;
```

Output:

book_id	title	price
2	Advanced SQL	650.00
3	Python Guide	550.00
4	Java Handbook	700.00
5	Data Science	900.00
6	Statistics 101	450.00
8	Cloud Computing	800.00

Query 6: ALL Operator – Display books priced above every Database book

Query:

```
SELECT book_id, title, price
FROM Books
WHERE price > ALL (
  SELECT price
  FROM Books
  WHERE category = 'Database'
)
ORDER BY book_id;
```

Output:

book_id	title	price
4	Java Handbook	700.00

5	Data Science	900.00
8	Cloud Computing	800.00

Query 7: EXISTS Operator – Display members who borrowed books

Query:

```
SELECT m.member_id, m.member_name
FROM Members m
WHERE EXISTS (
  SELECT 1
  FROM Loans l
  WHERE l.member_id = m.member_id
)
ORDER BY m.member_id;
```

Output:

member_id	member_name
101	Asha
102	Bala
103	Charan
104	Deepa

Query 8: NOT EXISTS Operator – Display members who have not borrowed books

Query:

```
SELECT m.member_id, m.member_name
FROM Members m
WHERE NOT EXISTS (
  SELECT 1
  FROM Loans l
  WHERE l.member_id = m.member_id
)
ORDER BY m.member_id;
```

Output:

member_id	member_name
105	Esha

Query 9: IN Subquery – Display borrowed book titles

Query:

```
SELECT book_id, title
FROM Books
WHERE book_id IN (
  SELECT book_id
  FROM Loans
)
ORDER BY book_id;
```

Output:

book_id	title
1	SQL Basics
2	Advanced SQL

3	Python Guide
4	Java Handbook
5	Data Science

Query 10: UNION Operator – Display CSE or AIML member names

Query:

```
SELECT member_name
FROM Members
WHERE department = 'CSE'
UNION
SELECT member_name
FROM Members
WHERE department = 'AIML'
ORDER BY member_name;
```

Output:

member_name
Asha
Charan
Deepa

Query 11: INTERSECT Operator – Display CSE members who borrowed books

Query:

```
SELECT member_id
FROM Members
WHERE department = 'CSE'
INTERSECT
SELECT member_id
FROM Loans
ORDER BY member_id;
```

Output:

member_id
101
103

Query 12: UNION Operator – Display books priced below 500 or above 750

Query:

```
SELECT title
FROM Books
WHERE price < 500
UNION
SELECT title
FROM Books WHERE
price > 750
ORDER BY title;
```

Output:

title
Cloud Computing

Data Science
SQL Basics
Statistics 101
Web Design

Query 13: INTERSECT Operator – Display Programming books that have been borrowed

Query:

```
SELECT book_id
FROM Books
WHERE category = 'Programming'
INTERSECT
SELECT book_id
FROM Loans
ORDER BY book_id;
```

Output:

book_id
3
4

Query 14: NOT EXISTS Operator – Display books that have never been borrowed

Query:

```
SELECT b.book_id, b.title
FROM Books b
WHERE NOT EXISTS (
  SELECT 1
  FROM Loans l
  WHERE l.book_id = b.book_id
)
ORDER BY b.book_id;
```

Output:

book_id	title
6	Statistics 101
7	Web Design
8	Cloud Computing

Query 15: CHECK Constraint Violation

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Books VALUES
  (9, 'Invalid Price', 'Test', -50, 2);
```

Output:

```
ERROR 3819 (HY000): Check constraint 'Books_chk_1' is violated.
```

Query 16: FOREIGN KEY Constraint Violation

This statement is intentionally invalid. The error shown is the expected result and no row is inserted.

Query:

```
INSERT INTO Loans VALUES
```

```
  (206, 999, 101, '2026-09-06');
```

Output:

```
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`LibraryExp5`.`Loans`, CONSTRAINT `Loans_ibfk_1` FOREIGN KEY (`book_id`) REFERENCES
`Books` (`book_id`))
```

Query 17: Verify Record Counts

Query:

```
SELECT
  (SELECT COUNT(*) FROM Books) AS books,
  (SELECT COUNT(*) FROM Members) AS members,
  (SELECT COUNT(*) FROM Loans) AS loans;
```

Output:

books	members	loans
8	5	5

Experiment 6

SQL Aggregate Functions, GROUP BY and HAVING

Aim: To create a retail sales database and execute MySQL queries using COUNT, SUM, AVG, MIN, MAX, DISTINCT, GROUP BY, HAVING, WHERE and ORDER BY to produce useful sales summaries.

Platform: MySQL 8.0+

Step 1: Create Database

Query:

```
CREATE DATABASE IF NOT EXISTS RetailSalesDB;
USE RetailSalesDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Step 2: Create Sales Table

Query:

```
CREATE TABLE Sales (
  Sale_ID INT PRIMARY KEY,
  Product_Name VARCHAR(40) NOT NULL,
  Category VARCHAR(25) NOT NULL,
  Region VARCHAR(20) NOT NULL,
  Quantity INT NOT NULL CHECK (Quantity > 0),
  Unit_Price DECIMAL(10,2) NOT NULL CHECK (Unit_Price >= 0) );
```

Output:

```
Query OK, 0 rows affected
```

Step 3: Insert 12 Records

Query:

```
INSERT INTO Sales VALUES
  (1, 'Laptop', 'Electronics', 'South', 2, 45000),
  (2, 'Mouse', 'Electronics', 'North', 10, 600),
  (3, 'Chair', 'Furniture', 'South', 4, 3500),
  (4, 'Desk', 'Furniture', 'East', 3, 8000),
  (5, 'Notebook', 'Stationery', 'West', 20, 80),
  (6, 'Pen', 'Stationery', 'North', 50, 20),
  (7, 'Monitor', 'Electronics', 'East', 5, 12000),
  (8, 'Chair', 'Furniture', 'North', 6, 3200),
  (9, 'Notebook', 'Stationery', 'South', 30, 75),
  (10, 'Keyboard', 'Electronics', 'West', 8, 1200),
  (11, 'Desk', 'Furniture', 'West', 2, 8500),
  (12, 'Pen', 'Stationery', 'East', 40, 25);
```

Output:

```
Query OK, 12 rows affected  Records: 12  Duplicates: 0  Warnings: 0
```

Query 1: COUNT – Total Sales Records

Query:

```
SELECT COUNT(*) AS Total_Records
FROM Sales;
```

Output:

Total_Records

12

Query 2: SUM – Total Units Sold

Query:

```
SELECT SUM(Quantity) AS Total_Units
FROM Sales;
```

Output:

Total_Units
180

Query 3: SUM – Total Sales Revenue

Query:

```
SELECT SUM(Quantity * Unit_Price) AS
Total_Revenue FROM Sales;
```

Output:

Total_Revenue
245650.00

Query 4: AVG – Average Unit Price

Query:

```
SELECT ROUND(AVG(Unit_Price), 2) AS
Avg_Price FROM Sales;
```

Output:

Avg_Price
6850.00

Query 5: MIN and MAX – Price Range

Query:

```
SELECT
  MIN(Unit_Price) AS Lowest_Price,
  MAX(Unit_Price) AS Highest_Price
FROM Sales;
```

Output:

Lowest_Price	Highest_Price
20.00	45000.00

Query 6: COUNT DISTINCT – Product Categories

Query:

```
SELECT COUNT(DISTINCT Category) AS Categories
FROM Sales;
```

Output:

Categories
3

Query 7: GROUP BY – Records per Category

Query:

```
SELECT Category, COUNT(*) AS Sale_Records
FROM Sales GROUP
BY Category
ORDER BY Category;
```

Output:

Category	Sale_Records
Electronics	4
Furniture	4
Stationery	4

Query 8: GROUP BY – Units per Category

Query:

```
SELECT Category, SUM(Quantity) AS Units_Sold
FROM Sales GROUP
BY Category
ORDER BY Category;
```

Output:

Category	Units_Sold
Electronics	25
Furniture	15
Stationery	140

Query 9: GROUP BY – Revenue per Category

Query:

```
SELECT Category,
  SUM(Quantity * Unit_Price) AS Revenue
FROM Sales GROUP
BY Category
ORDER BY Category;
```

Output:

Category	Revenue
Electronics	165600.00
Furniture	74200.00
Stationery	5850.00

Query 10: GROUP BY – Region-Wise Revenue

Query:

```
SELECT Region,
  SUM(Quantity * Unit_Price) AS Revenue
FROM Sales
GROUP BY Region
ORDER BY Revenue DESC, Region;
```

Output:

Region	Revenue

South	106250.00
East	85000.00
West	28200.00
North	26200.00

Query 11: HAVING – Categories with More Than 20 Units

Query:

```
SELECT Category,
  SUM(Quantity) AS Units_Sold
FROM Sales
GROUP BY Category HAVING
SUM(Quantity) > 20
ORDER BY Category;
```

Output:

Category	Units_Sold
Electronics	25
Stationery	140

Query 12: HAVING – Regions with Revenue Above 30000

Query:

```
SELECT Region,
  SUM(Quantity * Unit_Price) AS Revenue
FROM Sales
GROUP BY Region
HAVING SUM(Quantity * Unit_Price) > 30000
ORDER BY Revenue DESC;
```

Output:

Region	Revenue
South	106250.00
East	85000.00

Query 13: WHERE and HAVING – Filtered Aggregates

Query:

```
SELECT Category,
  SUM(Quantity) AS Units_Sold
FROM Sales
WHERE Region IN ('North', 'South')
GROUP BY Category HAVING
SUM(Quantity) >= 10
ORDER BY Category;
```

Output:

Category	Units_Sold
Electronics	12
Furniture	10
Stationery	80

Query 14: Multiple GROUP BY Columns

Query:

```
SELECT Category, Region,
  SUM(Quantity) AS Units_Sold
FROM Sales GROUP BY
Category, Region
ORDER BY Category, Region;
```

Output:

Category	Region	Units_Sold
Electronics	East	5
Electronics	North	10
Electronics	South	2
Electronics	West	8
Furniture	East	3
Furniture	North	6
Furniture	South	4
Furniture	West	2
Stationery	East	40
Stationery	North	50
Stationery	South	30
Stationery	West	20

Query 15: Top Category by Revenue

Query:

```
SELECT Category,
  SUM(Quantity * Unit_Price) AS Revenue
FROM Sales
GROUP BY Category
ORDER BY Revenue DESC, Category
LIMIT 1;
```

Output:

Category	Revenue
Electronics	165600.00

Query 16: Average Quantity by Category

Query:

```
SELECT Category,
  ROUND(AVG(Quantity), 2) AS Avg_Quantity
FROM Sales GROUP
BY Category
ORDER BY Category;
```

Output:

Category	Avg_Quantity
Electronics	6.25
Furniture	3.75

Stationery	35.00

Experiment 7

Creation of Views, Sequences, Indexes and Savepoints

Aim: To create and use database views, implement automatic sequential identifiers, create and inspect indexes, and control transactions using SAVEPOINT, ROLLBACK and COMMIT in MySQL.

Platform: MySQL 8.0+

Query 1: Create Database

Query:

```
CREATE DATABASE WarehouseDB;

USE WarehouseDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create Product Table

Query:

```
CREATE TABLE Product (
  Product_ID INT PRIMARY KEY AUTO_INCREMENT,
  Product_Name VARCHAR(60) NOT NULL,
Category VARCHAR(30) NOT NULL,
  Unit_Price DECIMAL(10,2) NOT NULL,
  Stock_Qty INT NOT NULL,
  Warehouse VARCHAR(30) NOT NULL
) ENGINE=InnoDB AUTO_INCREMENT=1001;
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Insert Eight Records

Query:

```
INSERT INTO Product
  (Product_Name, Category, Unit_Price, Stock_Qty, Warehouse)
VALUES
  ('Keyboard', 'Electronics', 800, 40, 'Chennai'),
  ('Mouse', 'Electronics', 450, 12, 'Chennai'),
  ('Office Chair', 'Furniture', 4500, 8, 'Bengaluru'),
  ('Desk', 'Furniture', 7200, 15, 'Chennai'),
  ('Notebook', 'Stationery', 60, 100, 'Hyderabad'),
  ('Pen Pack', 'Stationery', 120, 25, 'Hyderabad'),
  ('Monitor', 'Electronics', 12500, 6, 'Bengaluru'),
  ('Printer', 'Electronics', 9500, 4, 'Chennai');
```

Output:

```
Query OK, 8 rows affected  Records: 8  Duplicates: 0  Warnings: 0
```

Query 4: Create Product View

Query:

```
CREATE VIEW vw_product_catalog AS
SELECT Product_ID, Product_Name, Category, Unit_Price FROM
Product;
```

Output:

```
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Query 5: Display Product View

Query:

```
SELECT *
FROM vw_product_catalog
ORDER BY Product_ID;
```

Output:

Product_ID	Product_Name	Category	Unit_Price
1001	Keyboard	Electronics	800.00
1002	Mouse	Electronics	450.00
1003	Office Chair	Furniture	4500.00
1004	Desk	Furniture	7200.00
1005	Notebook	Stationery	60.00
1006	Pen Pack	Stationery	120.00
1007	Monitor	Electronics	12500.00
1008	Printer	Electronics	9500.00

Query 6: Create Low-Stock View

Query:

```
CREATE VIEW vw_low_stock AS
SELECT Product_ID, Product_Name, Stock_Qty
FROM Product
WHERE Stock_Qty < 10;
```

Output:

```
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Query 7: Display Low-Stock Products

Query:

```
SELECT * FROM
vw_low_stock
ORDER BY Product_ID;
```

Output:

Product_ID	Product_Name	Stock_Qty
1003	Office Chair	8
1007	Monitor	6
1008	Printer	4

Query 8: Replace Low-Stock View

Query:

```
CREATE OR REPLACE VIEW vw_low_stock AS
SELECT Product_ID, Product_Name, Category, Stock_Qty
FROM Product
WHERE Stock_Qty <= 12;

SELECT * FROM
vw_low_stock
ORDER BY Product_ID;
```

Output:

```
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Product_ID	Product_Name	Category	Stock_Qty
1002	Mouse	Electronics	12
1003	Office Chair	Furniture	8
1007	Monitor	Electronics	6
1008	Printer	Electronics	4

Query 9: Generate Automatic Product ID

Query:

```
INSERT INTO Product
  (Product_Name, Category, Unit_Price, Stock_Qty, Warehouse)
VALUES
  ('Webcam', 'Electronics', 2200, 18, 'Chennai');

SELECT LAST_INSERT_ID() AS New_Product_ID;
```

Output:

```
Query OK, 1 row affected
```

New_Product_ID
1009

Query 10: Create Index

Query:

```
CREATE INDEX idx_product_category
ON Product(Category);
```

Output:

```
Query OK, 0 rows affected
```

Query 11: Display Indexes

Query:

```
SHOW INDEX FROM Product;
```

Output:

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type	Visible
Product	0	PRIMARY	1	Product_ID	A	9	BTREE	YES
Product	1	idx_product_category	1	Category	A	3	BTREE	YES

Query 12: Explain Indexed Query

Query:

```
EXPLAIN
SELECT *
FROM Product
WHERE Category = 'Electronics';
```

Output:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	Product	ref	idx_product_category	idx_product_category	122	const	5	100.00	NULL

Query 13: Create and Use SAVEPOINT

Query:

```
START TRANSACTION;

UPDATE Product
SET Stock_Qty = Stock_Qty - 5
WHERE Product_ID = 1001;

SAVEPOINT sp_after_keyboard;

UPDATE Product
SET Stock_Qty = Stock_Qty - 3
WHERE Product_ID = 1002;

SELECT Product_ID, Stock_Qty
FROM Product
WHERE Product_ID IN (1001, 1002)
ORDER BY Product_ID;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Product_ID	Stock_Qty
1001	35
1002	9

Query 14: ROLLBACK TO SAVEPOINT

Query:

```
ROLLBACK TO SAVEPOINT sp_after_keyboard;

SELECT Product_ID, Stock_Qty
FROM Product
WHERE Product_ID IN (1001, 1002)
ORDER BY Product_ID;
```

Output:

Product_ID	Stock_Qty
1001	35
1002	12

Query 15: COMMIT Transaction

Query:

```
COMMIT;

SELECT Product_ID, Stock_Qty
FROM Product
WHERE Product_ID IN (1001, 1002)
ORDER BY Product_ID;
```

Output:

Product_ID	Stock_Qty
1001	35
1002	12

Query 16: Full ROLLBACK

Query:

```
START TRANSACTION;

UPDATE Product
SET Stock_Qty = 0
WHERE Product_ID = 1003;

ROLLBACK;

SELECT Product_ID, Stock_Qty
FROM Product
WHERE Product_ID = 1003;
```

Output:

```
Query OK, 1 row affected  Rows matched: 1  Changed: 1  Warnings: 0
```

Product_ID	Stock_Qty
1003	8

Query 17: Drop Index

Query:

```
DROP INDEX idx_product_category
  ON Product;

SHOW INDEX FROM Product;
```

Output:

```
Query OK, 0 rows affected
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Index_type	Visible
Product	0	PRIMARY	1	Product_ID	A	9	BTREE	YES

Experiment 8

Normalization (1NF, 2NF, 3NF, 4NF and 5NF) using Dependencies

Aim: To identify data redundancy and update anomalies, determine functional and multivalued dependencies, and normalize sample relations through First, Second, Third, Fourth and Fifth Normal Forms using MySQL.

Platform: MySQL 8.0+

Query 1: Create Database

Query:

```
CREATE DATABASE IF NOT EXISTS NormalizationLabDB;
USE NormalizationLabDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2: Create Unnormalized Table (UNF) – Create Table

Query:

```
CREATE TABLE EnrollmentRaw (
  StudentID INT,
  StudentName VARCHAR(40),
  CourseList VARCHAR(100)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 2: Create Unnormalized Table (UNF) – Insert Records

Query:

```
INSERT INTO EnrollmentRaw VALUES
  (1, 'Anu', 'C101,C102'),
  (2, 'Bala', 'C101'),
  (3, 'Chitra', 'C102');
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 2: Create Unnormalized Table (UNF) – Display Records

Query:

```
SELECT *
FROM EnrollmentRaw
ORDER BY StudentID;
```

Output:

StudentID	StudentName	CourseList
1	Anu	C101,C102
2	Bala	C101
3	Chitra	C102

Query 3: Convert to 1NF – Create Table

Query:

```
CREATE TABLE Enrollment1NF (
  StudentID INT,
  StudentName VARCHAR(40),
  CourseID VARCHAR(8),
  CourseName VARCHAR(40),
  InstructorID VARCHAR(8),
  InstructorName VARCHAR(40),
  Grade CHAR(1),
  PRIMARY KEY (StudentID, CourseID)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 3: Convert to 1NF – Insert Records

Query:

```
INSERT INTO Enrollment1NF VALUES
  (1, 'Anu', 'C101', 'DBMS', 'I10', 'Rao', 'A'),
  (1, 'Anu', 'C102', 'Networks', 'I20', 'Sen', 'B'),
  (2, 'Bala', 'C101', 'DBMS', 'I10', 'Rao', 'B'),
  (3, 'Chitra', 'C102', 'Networks', 'I20', 'Sen', 'A');
```

Output:

```
Query OK, 4 rows affected  Records: 4  Duplicates: 0  Warnings: 0
```

Query 3: Convert to 1NF – Display Records

Query:

```
SELECT StudentID, CourseID, Grade
FROM Enrollment1NF
ORDER BY StudentID, CourseID;
```

Output:

StudentID	CourseID	Grade
1	C101	A
1	C102	B
2	C101	B
3	C102	A

Query 4(a): Create Students2NF Table

Query:

```
CREATE TABLE Students2NF (
  StudentID INT PRIMARY KEY,
  StudentName VARCHAR(40) NOT NULL
);
```

Output:

```
Query OK, 0 rows affected
```

Query 4(b): Create Courses2NF Table

Query:

```
CREATE TABLE Courses2NF (
  CourseID VARCHAR(8) PRIMARY KEY,
  CourseName VARCHAR(40),
  InstructorID VARCHAR(8),
  InstructorName VARCHAR(40)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 4(c): Create Registrations2NF Table

Query:

```
CREATE TABLE Registrations2NF (
  StudentID INT,
  CourseID VARCHAR(8),
  Grade CHAR(1),
  PRIMARY KEY (StudentID, CourseID),
FOREIGN KEY (StudentID)
  REFERENCES Students2NF(StudentID),
  FOREIGN KEY (CourseID)
  REFERENCES Courses2NF(CourseID)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 4: Convert to 2NF – Insert Students

Query:

```
INSERT INTO Students2NF VALUES
  (1, 'Anu'),
```

```
  (2, 'Bala'),
  (3, 'Chitra');
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 4: Convert to 2NF – Insert Courses

Query:

```
INSERT INTO Courses2NF VALUES
  ('C101', 'DBMS', 'I10', 'Rao'),
  ('C102', 'Networks', 'I20', 'Sen');
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
```

Query 4: Convert to 2NF – Insert Registrations

Query:

```
INSERT INTO Registrations2NF VALUES
  (1, 'C101', 'A'),
  (1, 'C102', 'B'),
  (2, 'C101', 'B'),
  (3, 'C102', 'A');
```

Output:

```
Query OK, 4 rows affected  Records: 4  Duplicates: 0  Warnings: 0
```

Query 4: Convert to 2NF – Display Records

Query:

```
SELECT * FROM Students2NF
ORDER BY StudentID;

SELECT * FROM Courses2NF
ORDER BY CourseID;

SELECT * FROM Registrations2NF
ORDER BY StudentID, CourseID;
```

Output:

StudentID	StudentName
1	Anu
2	Bala
3	Chitra

CourseID	CourseName	InstructorID	InstructorName
C101	DBMS	I10	Rao
C102	Networks	I20	Sen

StudentID	CourseID	Grade
1	C101	A
1	C102	B
2	C101	B
3	C102	A

Query 5(a): Create Instructors3NF Table

Query:

```
CREATE TABLE Instructors3NF (
  InstructorID VARCHAR(8) PRIMARY KEY,
  InstructorName VARCHAR(40) NOT NULL
);
```

Output:

```
Query OK, 0 rows affected
```

Query 5(b): Create Courses3NF Table

Query:

```
CREATE TABLE Courses3NF (
  CourseID VARCHAR(8) PRIMARY KEY,
  CourseName VARCHAR(40) NOT NULL,
  InstructorID VARCHAR(8) NOT NULL,
  FOREIGN KEY (InstructorID)
  REFERENCES Instructors3NF(InstructorID)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 5: Convert to 3NF – Insert Instructors

Query:

```
INSERT INTO Instructors3NF VALUES
  ('I10', 'Rao'),
  ('I20', 'Sen');
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
```

Query 5: Convert to 3NF – Insert Courses

Query:

```
INSERT INTO Courses3NF VALUES
  ('C101', 'DBMS', 'I10'),
  ('C102', 'Networks', 'I20');
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
```

Query 5: Convert to 3NF – Display 3NF Records

Query:

```
SELECT r.StudentID,
s.StudentName,
  c.CourseName,
  i.InstructorName,
  r.Grade
FROM Registrations2NF r
JOIN Students2NF s
  ON s.StudentID = r.StudentID
JOIN Courses3NF c
  ON c.CourseID = r.CourseID
JOIN Instructors3NF i
  ON i.InstructorID = c.InstructorID
ORDER BY r.StudentID, c.CourseID; Output:
```

StudentID	StudentName	CourseName	InstructorName	Grade
1	Anu	DBMS	Rao	A
1	Anu	Networks	Sen	B
2	Bala	DBMS	Rao	B
3	Chitra	Networks	Sen	A

Query 6: Demonstrate 4NF Problem – Create Table

Query:

```
CREATE TABLE StudentInterestRaw (
  StudentID INT,
  Hobby VARCHAR(30),
  Language VARCHAR(30),
  PRIMARY KEY (StudentID, Hobby, Language) );
```

Output:

```
Query OK, 0 rows affected
```

Query 6: Demonstrate 4NF Problem – Insert Records

Query:

```
INSERT INTO StudentInterestRaw VALUES
  (1, 'Chess', 'English'),
  (1, 'Chess', 'Tamil'),
  (1, 'Music', 'English'),
  (1, 'Music', 'Tamil');
```

Output:

```
Query OK, 4 rows affected  Records: 4  Duplicates: 0  Warnings: 0
```

Query 6: Demonstrate 4NF Problem – Display Records

Query:

```
SELECT *
FROM StudentInterestRaw
ORDER BY Hobby, Language;
```

Output:

StudentID	Hobby	Language
1	Chess	English
1	Chess	Tamil
1	Music	English
1	Music	Tamil

Query 7(a): Create StudentHobby4NF Table

Query:

```
CREATE TABLE StudentHobby4NF (
  StudentID INT,
  Hobby VARCHAR(30),
  PRIMARY KEY (StudentID, Hobby)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 7(b): Create StudentLanguage4NF Table

Query:

```
CREATE TABLE StudentLanguage4NF (
  StudentID INT,
  Language VARCHAR(30),
  PRIMARY KEY (StudentID, Language)
);
```

Output:

```
Query OK, 0 rows affected
```

Query 7: Convert to 4NF – Insert Hobbies

Query:

```
INSERT INTO StudentHobby4NF VALUES
  (1, 'Chess'),
  (1, 'Music');
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
```

Query 7: Convert to 4NF – Insert Languages

Query:

```
INSERT INTO StudentLanguage4NF VALUES
  (1, 'English'),
  (1, 'Tamil');
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
```

Query 7: Convert to 4NF – Display Reconstructed Records

Query:

```
SELECT h.StudentID, h.Hobby, l.Language
FROM StudentHobby4NF h
JOIN StudentLanguage4NF l   ON
h.StudentID = l.StudentID
ORDER BY h.Hobby, l.Language;
```

Output:

StudentID	Hobby	Language
1	Chess	English
1	Chess	Tamil
1	Music	English
1	Music	Tamil

Query 8(a): Create SupplyRaw Table

Query:

```
CREATE TABLE SupplyRaw (
  SupplierID VARCHAR(8),
  PartID VARCHAR(8),
  ProjectID VARCHAR(8),
  PRIMARY KEY (SupplierID, PartID, ProjectID) );
```

Output:

```
Query OK, 0 rows affected
```

Query 8: Demonstrate 5NF – Insert Records

Query:

```
INSERT INTO SupplyRaw VALUES
  ('S1', 'P1', 'J1'),
  ('S1', 'P2', 'J1'),
  ('S2', 'P1', 'J1');
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
```

Query 8(b): Create SupplierPart5NF Table

Query:

```
CREATE TABLE SupplierPart5NF AS
SELECT DISTINCT SupplierID, PartID
FROM SupplyRaw;

ALTER TABLE SupplierPart5NF
ADD PRIMARY KEY (SupplierID, PartID);
```

Output:

```
Query OK, 3 rows affected  Records: 3  Duplicates: 0  Warnings: 0
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Query 8(c): Create SupplierProject5NF Table

Query:

```
CREATE TABLE SupplierProject5NF AS
SELECT DISTINCT SupplierID, ProjectID
FROM SupplyRaw;

ALTER TABLE SupplierProject5NF
ADD PRIMARY KEY (SupplierID, ProjectID);
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Query 8(d): Create PartProject5NF Table

Query:

```
CREATE TABLE PartProject5NF AS
SELECT DISTINCT PartID, ProjectID
FROM SupplyRaw;

ALTER TABLE PartProject5NF
ADD PRIMARY KEY (PartID, ProjectID);
```

Output:

```
Query OK, 2 rows affected  Records: 2  Duplicates: 0  Warnings: 0
Query OK, 0 rows affected  Records: 0  Duplicates: 0  Warnings: 0
```

Query 8: Demonstrate 5NF – Display Reconstructed Records

Query:

```
SELECT sp.SupplierID,
sp.PartID,
sj.ProjectID FROM
SupplierPart5NF sp
JOIN SupplierProject5NF sj
  ON sj.SupplierID = sp.SupplierID
JOIN PartProject5NF pj
  ON pj.PartID = sp.PartID
```

```
  AND pj.ProjectID = sj.ProjectID
ORDER BY sp.SupplierID, sp.PartID, sj.ProjectID;
```

Output:

SupplierID	PartID	ProjectID
S1	P1	J1
S1	P2	J1
S2	P1	J1

Query 9(a): Count Original Records

Query:

```
SELECT COUNT(*) AS OriginalRows
FROM SupplyRaw;
```

Output:

OriginalRows
3

Query 9(b): Count Reconstructed Records

Query:

```
SELECT COUNT(*) AS ReconstructedRows
FROM (
  SELECT sp.SupplierID,
sp.PartID,   sj.ProjectID   FROM
SupplierPart5NF sp
  JOIN SupplierProject5NF sj
  USING (SupplierID)
  JOIN PartProject5NF pj
  ON pj.PartID = sp.PartID
  AND pj.ProjectID = sj.ProjectID
) AS Reconstructed;
```

Output:

ReconstructedRows
3

Query 9(c): Verify No Differences

Query:

```
WITH Reconstructed AS (
SELECT sp.SupplierID,
sp.PartID,   sj.ProjectID
FROM SupplierPart5NF sp
  JOIN SupplierProject5NF sj
  USING (SupplierID)
  JOIN PartProject5NF pj
  ON pj.PartID = sp.PartID
  AND pj.ProjectID = sj.ProjectID
)
SELECT 'Original only' AS DifferenceType,
  o.SupplierID, o.PartID, o.ProjectID
FROM SupplyRaw o
LEFT JOIN Reconstructed r
  ON r.SupplierID = o.SupplierID
```

```
  AND r.PartID = o.PartID
  AND r.ProjectID = o.ProjectID
WHERE r.SupplierID IS NULL
UNION ALL
SELECT 'Reconstructed only' AS DifferenceType,
  r.SupplierID, r.PartID, r.ProjectID
FROM Reconstructed r
LEFT JOIN SupplyRaw o
  ON r.SupplierID = o.SupplierID
  AND r.PartID = o.PartID   AND
r.ProjectID = o.ProjectID
WHERE o.SupplierID IS NULL;
```

Output:

```
Empty set (0 rows)
```

Experiment 9

Creating Relationships Between Tables and Performing JOINs and Subqueries

Aim: To design related relational tables using primary and foreign keys, retrieve combined information through JOIN operations, and execute scalar, multi-row, correlated and nested subqueries in MySQL.

Platform: MySQL 8.0+

Query 1: Create Database

Query:

```
CREATE DATABASE IF NOT EXISTS CompanyDB;

USE CompanyDB;
```

Output:

```
Query OK, 1 row affected
Database changed
```

Query 2(a): Create Departments Table

Query:

```
CREATE TABLE Departments (
  Dept_ID INT PRIMARY KEY,
  Dept_Name VARCHAR(40) NOT NULL UNIQUE );
```

Output:

```
Query OK, 0 rows affected
```

Query 2(b): Create Employees Table

Query:

```
CREATE TABLE Employees (
  Emp_ID INT PRIMARY KEY,
  Emp_Name VARCHAR(40) NOT NULL,
Dept_ID INT NOT NULL,
  Salary DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID) );
```

Output:

```
Query OK, 0 rows affected
```

Query 2(c): Create Projects Table

Query:

```
CREATE TABLE Projects (
  Project_ID INT PRIMARY KEY,
  Project_Name VARCHAR(60) NOT NULL,
  Dept_ID INT NOT NULL,
  Budget DECIMAL(12,2) NOT NULL,
  FOREIGN KEY (Dept_ID) REFERENCES Departments(Dept_ID) );
```

Output:

```
Query OK, 0 rows affected
```

Query 2(d): Create Assignments Table

Query:

```
CREATE TABLE Assignments (
Emp_ID INT NOT NULL,
  Project_ID INT NOT NULL,
  Hours INT NOT NULL,
  PRIMARY KEY (Emp_ID, Project_ID),
  FOREIGN KEY (Emp_ID) REFERENCES Employees(Emp_ID),
  FOREIGN KEY (Project_ID) REFERENCES Projects(Project_ID) );
```

Output:

```
Query OK, 0 rows affected
```

Query 3(a): Insert Departments

Query:

```
INSERT INTO Departments VALUES
  (10, 'Development'),
  (20, 'Testing'),
  (30, 'HR'),
  (40, 'Research');
```

Output:

```
Query OK, 4 rows affected  Records: 4  Duplicates: 0  Warnings: 0
```

Query 3(b): Insert Employees

Query:

```
INSERT INTO Employees VALUES
(101, 'Arun', 10, 60000),
  (102, 'Priya', 10, 75000),
(103, 'Ravi', 20, 50000),
  (104, 'Sneha', 20, 55000),
  (105, 'Kiran', 30, 45000),
  (106, 'Divya', 40, 90000),
  (107, 'Rahul', 10, 65000);
```

Output:

```
Query OK, 7 rows affected  Records: 7  Duplicates: 0  Warnings: 0
```

Query 3(c): Insert Projects

Query:

```
INSERT INTO Projects VALUES
  (201, 'Web Portal', 10, 300000),
  (202, 'Mobile App', 10, 450000),
  (203, 'QA Automation', 20, 200000),
  (204, 'AI Prototype', 40, 600000),
  (205, 'Recruitment Portal', 30, 120000);
```

Output:

```
Query OK, 5 rows affected  Records: 5  Duplicates: 0  Warnings: 0
```

Query 3(d): Insert Assignments

Query:

```
INSERT INTO Assignments VALUES
  (101, 201, 30),
  (102, 201, 25),
  (102, 202, 35),
  (103, 203, 40),
  (104, 203, 20),
  (106, 204, 45),
  (107, 202, 30);
```

Output:

```
Query OK, 7 rows affected  Records: 7  Duplicates: 0  Warnings: 0
```

Query 4: INNER JOIN – Employees and Departments

Query:

```
SELECT e.Emp_Name, d.Dept_Name
FROM Employees e
INNER JOIN Departments d
  ON e.Dept_ID = d.Dept_ID
ORDER BY e.Emp_ID;
```

Output:

Emp_Name	Dept_Name
Arun	Development
Priya	Development
Ravi	Testing
Sneha	Testing
Kiran	HR
Divya	Research
Rahul	Development

Query 5: JOIN Three Tables – Project Assignments

Query:

```
SELECT e.Emp_Name, p.Project_Name, a.Hours
FROM Assignments a
JOIN Employees e
  ON a.Emp_ID = e.Emp_ID
JOIN Projects p
  ON a.Project_ID = p.Project_ID
ORDER BY a.Emp_ID, a.Project_ID;
```

Output:

Emp_Name	Project_Name	Hours
Arun	Web Portal	30
Priya	Web Portal	25
Priya	Mobile App	35
Ravi	QA Automation	40
Sneha	QA Automation	20

Divya	AI Prototype	45
Rahul	Mobile App	30

Query 6: LEFT JOIN – All Employees

Query:

```
SELECT e.Emp_Name, p.Project_Name
FROM Employees e
LEFT JOIN Assignments a
  ON e.Emp_ID = a.Emp_ID
LEFT JOIN Projects p
  ON a.Project_ID = p.Project_ID
ORDER BY e.Emp_ID, p.Project_ID;
```

Output:

Emp_Name	Project_Name
Arun	Web Portal
Priya	Web Portal
Priya	Mobile App
Ravi	QA Automation
Sneha	QA Automation
Kiran	NULL
Divya	AI Prototype
Rahul	Mobile App

Query 7: RIGHT JOIN – All Projects

Query:

```
SELECT p.Project_Name, e.Emp_Name
FROM Assignments a
RIGHT JOIN Projects p
  ON a.Project_ID = p.Project_ID
LEFT JOIN Employees e
ON a.Emp_ID = e.Emp_ID
ORDER BY p.Project_ID, e.Emp_ID;
```

Output:

Project_Name	Emp_Name
Web Portal	Arun
Web Portal	Priya
Mobile App	Priya
Mobile App	Rahul
QA Automation	Ravi
QA Automation	Sneha
AI Prototype	Divya
Recruitment Portal	NULL

Query 8: CROSS JOIN – Department and Project Combinations

Query:

```
SELECT d.Dept_Name, p.Project_Name
FROM Departments d
CROSS JOIN Projects p
WHERE d.Dept_ID = 30
ORDER BY p.Project_ID;
```

Output:

Dept_Name	Project_Name
HR	Web Portal
HR	Mobile App
HR	QA Automation
HR	AI Prototype
HR	Recruitment Portal

Query 9: SELF JOIN – Employees in Same Department

Query:

```
SELECT a.Emp_Name AS Employee1,
  b.Emp_Name AS Employee2
FROM Employees a
JOIN Employees b
  ON a.Dept_ID = b.Dept_ID
  AND a.Emp_ID < b.Emp_ID
ORDER BY a.Emp_ID, b.Emp_ID;
```

Output:

Employee1	Employee2
Arun	Priya
Arun	Rahul
Priya	Rahul
Ravi	Sneha

Query 10: Scalar Subquery – Salary Above Company Average

Query:

```
SELECT Emp_Name, Salary
FROM Employees
WHERE Salary > (
  SELECT AVG(Salary)
  FROM Employees
)
ORDER BY Salary DESC;
```

Output:

Emp_Name	Salary
Divya	90000.00
Priya	75000.00
Rahul	65000.00

Query 11: IN Subquery – Project-Owning Departments

Query:

```
SELECT Emp_Name, Dept_ID
FROM Employees
WHERE Dept_ID IN (
  SELECT Dept_ID
  FROM Projects
  WHERE Budget >= 400000
)
ORDER BY Emp_ID;
```

Output:

Emp_Name	Dept_ID
Arun	10
Priya	10
Divya	40
Rahul	10

Query 12: EXISTS – Assigned Employees

Query:

```
SELECT e.Emp_Name
FROM Employees e
WHERE EXISTS (
  SELECT 1
  FROM Assignments a
  WHERE a.Emp_ID = e.Emp_ID
)
ORDER BY e.Emp_ID;
```

Output:

Emp_Name
Arun
Priya
Ravi
Sneha
Divya
Rahul

Query 13: NOT EXISTS – Unassigned Employees

Query:

```
SELECT e.Emp_Name
FROM Employees e
WHERE NOT EXISTS (
  SELECT 1
  FROM Assignments a
  WHERE a.Emp_ID = e.Emp_ID
);
```

Output:

Emp_Name

Kiran

Query 14: Correlated Subquery – Above Department Average

Query:

```
SELECT e.Emp_Name, e.Dept_ID, e.Salary
FROM Employees e
WHERE e.Salary > (
  SELECT AVG(x.Salary)
  FROM Employees x
  WHERE x.Dept_ID = e.Dept_ID
)
ORDER BY e.Emp_ID;
```

Output:

Emp_Name	Dept_ID	Salary
Priya	10	75000.00
Sneha	20	55000.00

Query 15: MAX Subquery – Highest-Paid Employee

Query:

```
SELECT Emp_Name, Salary
FROM Employees
WHERE Salary = (
  SELECT MAX(Salary)
  FROM Employees
);
```

Output:

Emp_Name	Salary
Divya	90000.00

Query 16: JOIN and GROUP BY – Project Staffing

Query:

```
SELECT p.Project_Name,
  COUNT(a.Emp_ID) AS Staff
FROM Projects p
LEFT JOIN Assignments a
  ON p.Project_ID = a.Project_ID
GROUP BY p.Project_ID, p.Project_Name
ORDER BY p.Project_ID;
```

Output:

Project_Name	Staff
Web Portal	2
Mobile App	2
QA Automation	2
AI Prototype	1
Recruitment Portal	0

Query 17: NOT EXISTS – Projects Without Staff

Query:

```
SELECT p.Project_Name
FROM Projects p
WHERE NOT EXISTS (
  SELECT 1
  FROM Assignments a
  WHERE a.Project_ID = p.Project_ID
);
```

Output:

Project_Name
Recruitment Portal

Experiment 10

PL/SQL Programming: Conditional Blocks, Exceptions, Procedures, Functions, Triggers and Cursors

Aim: To write and execute Oracle PL/SQL programs that accept user input, implement conditions, handle exceptions, create stored procedures and functions, implement INSERT/UPDATE/DELETE triggers, and process rows using implicit and explicit cursors.

Platform: Oracle Database with SQL Developer / SQL*Plus

Query 1: Create Patients Table

Query:

```
CREATE TABLE patients (   patient_id
NUMBER(5) PRIMARY KEY,   patient_name
VARCHAR2(40) NOT NULL,   department
VARCHAR2(30) NOT NULL,
  bill_amount NUMBER(10,2) CHECK (bill_amount >= 0),
paid_amount NUMBER(10,2) DEFAULT 0 NOT NULL,
  CONSTRAINT ck_paid CHECK (
paid_amount >= 0
    AND paid_amount <= bill_amount
  )
);
```

Output:

```
Table PATIENTS created.
```

Query 2: Create Patient Audit Table

Query:

```
CREATE TABLE patient_audit (
  audit_id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
patient_id NUMBER(5),   action_type VARCHAR2(10),
old_bill NUMBER(10,2),   new_bill NUMBER(10,2),
  changed_on TIMESTAMP DEFAULT SYSTIMESTAMP );
```

Output:

```
Table PATIENT_AUDIT created.
```

Query 3: Insert Five Records

Query:

```
INSERT INTO patients VALUES
```

```
  (201, 'Asha', 'Cardiology', 12000, 8000); INSERT
INTO patients VALUES
  (202, 'Bala', 'Orthopedics', 9000, 9000);
INSERT INTO patients VALUES
  (203, 'Charan', 'Neurology', 18000, 6000);
INSERT INTO patients VALUES
  (204, 'Deepa', 'Cardiology', 7000, 3000);
INSERT INTO patients VALUES
  (205, 'Eshan', 'General', 5000, 1000); COMMIT;
```

Output:

```
1 row inserted.  (× 5) Commit
complete.
```

Query 4: Display Patient Balances

Query:

```
SELECT patient_id,
patient_name,
  bill_amount - paid_amount AS balance
FROM patients
ORDER BY patient_id;
```

Output:

PATIENT_ID	PATIENT_NAME	BALANCE
201	Asha	4000
202	Bala	0
203	Charan	12000
204	Deepa	4000
205	Eshan	4000

Query 5: Conditional PL/SQL Block (Classify Patient Bills)

*ACCEPT is SQL*Plus / SQL Developer script syntax (run with F5). The block is executed three times with different patient IDs.*

Query:

```
SET SERVEROUTPUT ON;
ACCEPT p_id NUMBER PROMPT 'Enter patient ID: '
DECLARE   v_id patients.patient_id%TYPE :=
&p_id;   v_name patients.patient_name%TYPE;
  v_due NUMBER(10,2);
BEGIN
  SELECT patient_name,
bill_amount - paid_amount
  INTO v_name, v_due
  FROM patients
  WHERE patient_id = v_id;
  IF v_due = 0 THEN
    DBMS_OUTPUT.PUT_LINE(v_name || ': Fully paid');
  ELSIF v_due > 10000 THEN
    DBMS_OUTPUT.PUT_LINE(v_name || ': High outstanding');
ELSE
    DBMS_OUTPUT.PUT_LINE(v_name || ': Payment pending');
  END IF;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    DBMS_OUTPUT.PUT_LINE('Patient not found');
END;
```

```
/
```

Output:

Input: patient ID 203

```
Enter patient ID: 203
Charan: High outstanding

PL/SQL procedure successfully completed.
```

Input: patient ID 202

```
Enter patient ID: 202
Bala: Fully paid

PL/SQL procedure successfully completed.
```

Input: patient ID 999

```
Enter patient ID: 999
Patient not found

PL/SQL procedure successfully completed.
```

Query 6: Exception Handling (NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE)

Query:

```
DECLARE
  v_name patients.patient_name%TYPE;
  v_result NUMBER;
BEGIN
  BEGIN
    SELECT patient_name
    INTO v_name
    FROM patients
    WHERE patient_id = 999;
  EXCEPTION
    WHEN NO_DATA_FOUND THEN
      DBMS_OUTPUT.PUT_LINE('NO_DATA_FOUND handled');
  END;
  BEGIN
    SELECT patient_name
    INTO v_name
    FROM patients
    WHERE department = 'Cardiology';
  EXCEPTION
    WHEN TOO_MANY_ROWS THEN
      DBMS_OUTPUT.PUT_LINE('TOO_MANY_ROWS handled');
  END;
BEGIN
    v_result := 10 / 0;
  EXCEPTION
    WHEN ZERO_DIVIDE THEN
      DBMS_OUTPUT.PUT_LINE('ZERO_DIVIDE handled');
  END;
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Unexpected: ' || SQLERRM);
END;
/
```

Output:

```
NO_DATA_FOUND handled
TOO_MANY_ROWS handled
ZERO_DIVIDE handled
```

```
PL/SQL procedure successfully completed.
```

Query 7: Stored Procedure – Record Payment

Query:

```
CREATE OR REPLACE PROCEDURE record_payment (
p_patient_id IN NUMBER,   p_amount IN NUMBER
) AS
BEGIN
  IF p_amount <= 0 THEN
    RAISE_APPLICATION_ERROR(
      -20001, 'Payment must be positive'
    );
  END IF;
  UPDATE patients
  SET paid_amount = paid_amount + p_amount
  WHERE patient_id = p_patient_id
    AND paid_amount + p_amount <= bill_amount;
  IF SQL%ROWCOUNT = 0 THEN
    RAISE_APPLICATION_ERROR(
      -20002, 'Patient missing or payment exceeds balance'
    );
  END IF;
  DBMS_OUTPUT.PUT_LINE('Payment recorded');
END;
/
 BEGIN
  record_payment(201, 1000);
END;
/

SELECT paid_amount
FROM patients
WHERE patient_id = 201;

ROLLBACK;

SELECT paid_amount
FROM patients
WHERE patient_id = 201;
```

Output:

```
Procedure RECORD_PAYMENT compiled
Payment recorded

PL/SQL procedure successfully completed.
```

Before ROLLBACK

PAID_AMOUNT
9000

```
Rollback complete.
```

After ROLLBACK

PAID_AMOUNT
8000

The last SELECT (after ROLLBACK) was added to prove the payment was undone.

Query 8: Stored Function – Outstanding Balance

Query:

```
CREATE OR REPLACE FUNCTION get_balance (
p_patient_id IN NUMBER
)
RETURN NUMBER AS
v_balance NUMBER(10,2);
BEGIN
  SELECT bill_amount - paid_amount
  INTO v_balance
  FROM patients
  WHERE patient_id = p_patient_id;
  RETURN v_balance;
EXCEPTION
  WHEN NO_DATA_FOUND THEN
    RETURN NULL;
END;
/

SELECT patient_id,
  get_balance(patient_id) AS balance
FROM patients
ORDER BY patient_id;
```

Output:

```
Function GET_BALANCE compiled
```

PATIENT_ID	BALANCE
201	4000
202	0
203	12000
204	4000
205	4000

Query 9: INSERT Trigger

Query:

```
CREATE OR REPLACE TRIGGER trg_patient_insert
AFTER INSERT ON patients
FOR EACH ROW
BEGIN
  INSERT INTO patient_audit (
    patient_id, action_type, old_bill, new_bill
  )
  VALUES (
    :NEW.patient_id, 'INSERT',
    NULL,
    :NEW.bill_amount
  );
END;
/
```

Output:

```
Trigger TRG_PATIENT_INSERT compiled
```

Query 10: UPDATE Trigger

Query:

```
CREATE OR REPLACE TRIGGER trg_patient_update
AFTER UPDATE ON patients
FOR EACH ROW
BEGIN
  INSERT INTO patient_audit (
    patient_id, action_type, old_bill, new_bill
  )
  VALUES (
    :NEW.patient_id, 'UPDATE',
    :OLD.bill_amount,
    :NEW.bill_amount
  );
END;
/
```

Output:

```
Trigger TRG_PATIENT_UPDATE compiled
```

Query 11: DELETE Trigger

Query:

```
CREATE OR REPLACE TRIGGER trg_patient_delete
AFTER DELETE ON patients
FOR EACH ROW
BEGIN
  INSERT INTO patient_audit (
    patient_id, action_type, old_bill, new_bill
  )
  VALUES (
    :OLD.patient_id, 'DELETE',
    :OLD.bill_amount,
    NULL
  );
END;
/
```

Output:

```
Trigger TRG_PATIENT_DELETE compiled
```

Query 12: Test All Three Triggers

Query:

```
SAVEPOINT before_trigger_test;
INSERT INTO patients VALUES
  (206, 'Farah', 'General', 4000, 1000);
UPDATE patients
SET bill_amount = 4500
WHERE patient_id = 206; DELETE
FROM patients
WHERE patient_id = 206;
SELECT patient_id,
action_type,
old_bill,   new_bill
FROM patient_audit
ORDER BY audit_id;
ROLLBACK TO before_trigger_test;
```

Output:

```
Savepoint
created.
1 row inserted.
1 row updated.
1 row deleted.
```

PATIENT_ID	ACTION_TYPE	OLD_BILL	NEW_BILL

206	INSERT	(null)	4000
206	UPDATE	4000	4500
206	DELETE	4500	(null)

```
Rollback complete.
```

Query 13: Implicit Cursor (SQL%ROWCOUNT)

Query:

```
BEGIN
  UPDATE patients
  SET paid_amount = paid_amount + 500
  WHERE department = 'Cardiology'
    AND paid_amount + 500 <= bill_amount;
  DBMS_OUTPUT.PUT_LINE( 'Updated rows: ' || SQL%ROWCOUNT
  );
  ROLLBACK;
END;
/
```

Output:

```
Updated rows: 2

PL/SQL procedure successfully completed.
```

Query 14: Explicit Cursor

Query:

```
DECLARE
  CURSOR c_due IS     SELECT
patient_name,       bill_amount -
paid_amount AS due     FROM patients
    WHERE bill_amount > paid_amount
ORDER BY patient_id;   v_name
patients.patient_name%TYPE;   v_due
NUMBER(10,2); BEGIN
  OPEN c_due;
  LOOP
    FETCH c_due INTO v_name, v_due;
    EXIT WHEN c_due%NOTFOUND;
DBMS_OUTPUT.PUT_LINE(
v_name || ': ' || v_due
    );
  END LOOP;
  CLOSE c_due; END;
/
```

Output:

```
Asha: 4000
Charan: 12000
Deepa: 4000
Eshan: 4000

PL/SQL procedure successfully completed.
```

Query 15: Final Verification

Query:

```
SELECT patient_id,
patient_name,
bill_amount,   paid_amount,
  bill_amount - paid_amount AS balance
```

```
FROM patients
ORDER BY patient_id;

SELECT COUNT(*) AS total_patients
FROM patients;

SELECT COUNT(*) AS audit_records
FROM patient_audit;
```

Output:

PATIENT_ID	PATIENT_NAME	BILL_AMOUNT	PAID_AMOUNT	BALANCE
201	Asha	12000	8000	4000
202	Bala	9000	9000	0
203	Charan	18000	6000	12000
204	Deepa	7000	3000	4000
205	Eshan	5000	1000	4000

TOTAL_PATIENTS
5

AUDIT_RECORDS
0